
Golf Course Analyzer

Memoria del Proyecto

Visión por Computador aplicada al análisis de campos de golf

EfficientNet-B0

PyTorch

OpenCV

Streamlit

Transfer Learning

albumentations

Python 3.14

Gonzalo Solans del Carre Díaz-Gálvez · Abril 2026

Índice de contenidos

- 1. Introducción y objetivo del proyecto
- 2. El dataset: las imágenes
- 3. Arquitectura general: el pipeline de 5 etapas
- 4. Etapa 1 — Organización del dataset
- 5. Etapa 2 — Preprocesado y etiquetado automático
- 6. Etapa 3 — Entrenamiento de la red neuronal
- 7. Etapa 4 — Inferencia y post-procesado
- 8. Etapa 5 — Aplicación web interactiva
- 9. Problemas encontrados y soluciones aplicadas
- 10. Resumen de mejoras implementadas
- 11. Resultados obtenidos

1. Introducción y objetivo del proyecto

El objetivo de este proyecto es desarrollar un sistema de visión por computador capaz de analizar fotografías aéreas de campos de golf y determinar automáticamente el estado del terreno: qué zonas tienen hierba sana, cuáles están estresadas, dónde hay bunkers y dónde hay árboles.

Esto tiene una aplicación directa muy concreta: un greenkeeper —el encargado de mantener un campo de golf— puede subir una fotografía tomada con un dron o satélite y obtener en segundos un diagnóstico visual de todo el campo, con métricas de salud, mapas de colores y recomendaciones de actuación.

El sistema es completamente automático: no requiere que nadie etiquete las imágenes a mano. El propio programa aprende a identificar las zonas usando técnicas de análisis de color y una red neuronal entrenada sobre los datos.

2. El dataset: las imágenes

Se utilizaron 3.370 ortofotografías de campos de golf daneses y unas 1200 de campos españoles, por desgracia no hay subidas en internet muchas imágenes para coger de referencia. Una ortofotografía es una fotografía aérea tomada desde arriba (dron o satélite) y corregida geométricamente para que sea como un mapa: sin distorsión de perspectiva, perfectamente plana.

Las imágenes contienen las siguientes clases de terreno:

Hierba sana: verde viva, bien irrigada y cortada.

Hierba estresada: amarillenta o parduzca, con déficit de nutrientes o estrés por sequía.

Bunkers: zonas de arena, color beige/arena claro.

Árboles: vegetación perimetral y grupos de árboles dentro del recorrido.

Otros: caminos, edificios, aparcamientos, etc.

Las imágenes se almacenaron en la carpeta `data/raw/archive/` y el sistema las encuentra automáticamente aunque estuvieran organizadas en subcarpetas.

3. Arquitectura general: el pipeline de 5 etapas

El proyecto se divide en 5 etapas que se ejecutan en orden. Cada etapa toma la salida de la anterior como entrada.

Etapas	Nombre	Script	Descripción
1	Organización del dataset	stage1_organize.py	Valida, deduplica y divide en train/val/test
2	Preprocesado y etiquetado	stage2_preprocess.py	Trocea en patches y etiqueta automáticamente por color HSV
3	Entrenamiento	stage3_train.py	La red neuronal aprende a clasificar cada patch
4	Inferencia y post-procesado	stage4_inference.py	Analiza imágenes nuevas y genera mapas de segmentación
5	Aplicación web	stage5_app.py	Interfaz visual para subir imágenes y ver resultados

Para ejecutar todo el pipeline desde cero:

```
python3 run_pipeline.py      # ejecuta etapas 1, 2 y 3
python3 run_pipeline.py 5    # lanza la app web
```

4. Etapa 1 – Organización del dataset

Fichero: src/stage1_organize.py

4.1 Validación

Comprueba que cada imagen se puede leer correctamente y que tiene un tamaño mínimo de 100×100 píxeles. Las imágenes corruptas o demasiado pequeñas se descartan automáticamente.

4.2 Eliminación de duplicados

Usa un algoritmo de perceptual hash (pHash): convierte cada imagen en una huella digital basada en su contenido visual. Si dos imágenes tienen la misma huella, una se descarta. Esto evita que el modelo aprenda dos veces las mismas fotos y se sesgue.

4.3 División en conjuntos

Las imágenes únicas y válidas se dividen aleatoriamente en tres grupos: 70 % para entrenamiento, 15 % para validación y 15 % para test. La división se guarda en data/splits.json. Con el dataset disponible: 3.370 imágenes encontradas, 0 inválidas y algunas duplicadas eliminadas.

5. Etapa 2 — Preprocesado y etiquetado automático

Fichero: src/stage2_preprocess.py. Esta es la etapa más importante porque resuelve el mayor problema de cualquier proyecto de visión por computador: el etiquetado. El proyecto usa supervisión débil: en lugar de etiquetar a mano, se etiqueta automáticamente usando el color de cada zona.

5.1 Trocear las imágenes en patches

Cada imagen (512×512 píxeles) se divide en pequeños trozos de 64×64 píxeles, con solapamiento de 32 píxeles entre patches consecutivos. Esto genera miles de pequeñas imágenes para entrenar.

5.2 Etiquetado automático por color HSV

Cada patch se convierte al espacio HSV (H: tono 0-180, S: saturación 0-255, V: brillo 0-255). Se asigna la etiqueta de la clase cuyo rango HSV coincide con al menos el 55 %

de los píxeles del patch:

Clase	Rango H	Rango S	Rango V	Descripción
healthy_grass	30-90	40-255	25-200	Verde con saturación visible
stressed_grass	18-38	55-235	60-235	Amarillo-naranja, S alto
bunker	10-35	5-90	140-255	Beige claro, poca saturación
other	—	—	—	No cumple ninguna clase anterior

5.3 Cambios en los rangos HSV

Problema: hierba marcada como estresada cuando estaba sana. El rango de stressed_grass tenía $S_{min}=25$, demasiado bajo; las sombras también encajaban.

Solución: S_{min} subido a 55 para exigir un tono amarillo-naranja real.

5.4 Aumentación de datos

Para robustecer el modelo, cada patch de entrenamiento se duplica con variaciones: volteo horizontal, volteo vertical, rotación, cambios de brillo/contraste, variaciones de color, ruido gaussiano y desenfoque.

6. Etapa 3 — Entrenamiento de la red neuronal

Fichero: src/stage3_train.py

6.1 La red neuronal: EfficientNet-B0

El modelo usado es EfficientNet-B0, una CNN diseñada por Google, pequeña y eficiente. Por lo que he investigado fue diseñada originalmente para clasificar 1.000 categorías de ImageNet. Aquí se adapta mediante transfer learning: se aprovecha el conocimiento previo de la red (detección de bordes, texturas, formas) y se reemplaza solo la última capa para clasificar las 5 categorías del proyecto.

6.2 El proceso de entrenamiento

El entrenamiento se divide en epochs (épocas). En cada pasada completa por todos los patches: la red predice la clase de cada patch, se calcula el error (loss) y los parámetros se ajustan para reducirlo. El máximo es de 25 epochs, con early stopping de 6 epochs consecutivos sin mejora en validación. El mejor modelo se guarda en `models/best_model.pth`.

6.3 Problema técnico durante el entrenamiento

Problema: al iniciar el entrenamiento apareció un error de certificados SSL al intentar descargar los pesos preentrenados de EfficientNet.

Solución: instalar los certificados SSL de Python ejecutando el script de instalación de certificados incluido con Python. Esto instala los certificados sin cambiar ninguna configuración del sistema operativo.

6.4 Resultados del entrenamiento por clase

Clase	Precisión	Recall	F1-Score
healthy_grass	80 %	96 %	87 %
stressed_grass	85 %	88 %	86 %
bunker	41 %	88 %	56 %
other	100 %	93 %	97 %

El modelo funciona bien en las clases principales. El punto débil es el bunker (41 % de precisión), que se confunde con caminos y otras superficies claras. Esta limitación se aborda en la Etapa 4 mediante el refinamiento por forma geométrica.

7. Etapa 4 — Inferencia y post-procesado

Fichero: `src/stage4_inference.py`

7.1 Sliding window inference

La imagen se divide en patches solapados igual que en el entrenamiento. El modelo clasifica cada patch y las probabilidades se acumulan en un mapa. Donde los patches se solapan, se promedian las probabilidades. El resultado es un mapa de segmentación donde cada píxel tiene asignada una clase.

7.2 CLAHE: normalización de iluminación

Problema detectado: en imágenes con zonas de sol y sombra, la hierba iluminada directamente aparece más amarilla/blanquecina y el modelo la confundía con hierba estresada.

Solución: aplicar CLAHE (Contrast Limited Adaptive Histogram Equalization) antes de pasar la imagen al modelo. Esta técnica iguala la iluminación de forma local, dividiendo la imagen en zonas pequeñas y ecualizando el brillo de cada una por separado. Así las zonas de sol no parecen más brillantes que las de sombra.

7.3 Refinamiento por forma de bunkers

Problema detectado: el modelo detectaba como bunker zonas que en realidad son caminos, aparcamientos o tejados (superficies claras y planas). La precisión del bunker es baja (41 %).

Solución: los bunkers reales siempre tienen bordes redondeados u ovalados, mientras que los caminos tienen formas rectangulares. Tras la segmentación, se analizan los contornos de las zonas detectadas como bunker: si el contorno tiene menos de 6 vértices al aproximarse con `approxPolyDP`, la forma es demasiado angular y se reclasifica como 'other'. Las regiones menores de 500 píxeles se descartan como ruido.

7.4 Corrección de franjas de corte

Problema detectado: en campos cortados en franjas alternas, el modelo detectaba las franjas más oscuras como hierba estresada aunque toda la hierba estuviera perfectamente sana.

Solución (`smooth_mowing_stripes()`): se identifican todas las regiones de hierba estresada como grupos de píxeles conectados (componentes conexas). Para cada grupo pequeño (menos de 8.000 píxeles), si más del 55 % de los píxeles vecinos son hierba sana, la región se reclasifica como sana. Una franja de corte siempre está rodeada de hierba sana; una zona genuinamente estresada suele ser grande y estar rodeada de más estrés.

7.5 Detección adaptativa de árboles

Problema detectado: los árboles eran clasificados como 'other' o incluso como 'stressed_grass' (especialmente en otoño), penalizando artificialmente el health score del campo.

Se probaron varias versiones hasta llegar a la solución definitiva:

Versión 1 — Solo color: buscaba píxeles verdes oscuros con rangos HSV fijos. Fallaba con árboles amarillentos.

Versión 2 — Color OR textura: demasiados falsos positivos.

Versión 3 — Color AND textura: demasiado estricto, muchos árboles no se detectaban.

Versión 4 (solución final) — Detección adaptativa: (1) detecta semillas con criterios estrictos (color Y textura), (2) aprende el color HSV promedio de esas semillas, (3) expande la detección a píxeles dentro de $\pm 1,5$ desviaciones típicas del color aprendido, (4) descarta regiones menores de 1.200 píxeles.

La ventaja de este enfoque es que se adapta a cada imagen sin necesidad de cambiar parámetros. Si los árboles son verdes oscuros, aprende eso; si son amarillentos por el otoño, también.

7.6 Mapa de confianza

Además del mapa de segmentación, se genera un mapa de confianza que muestra en qué zonas el modelo está seguro de su predicción y en cuál duda. Se calcula tomando la probabilidad máxima de cada píxel entre todas las clases: si el modelo asigna 95 % a 'hierba sana', la confianza es alta (verde); si el máximo es solo 40 %, el modelo duda (azul/morado).

7.7 Métricas calculadas

Health Score (0-100): porcentaje de hierba sana sobre el total de hierba (sana + estresada).

Nivel de estrés: Bajo (>80), Moderado (55-80), Alto (<55).

Stress Index: porcentaje de la hierba visible que está estresada.

Uniformidad: basada en la varianza del canal verde en zonas de hierba.

Distribución de zonas: porcentaje del área de la imagen que ocupa cada clase.

7.8 Recomendaciones automáticas

El sistema genera recomendaciones en texto según las métricas: hierba estresada >20 % sugiere revisar el sistema de riego; hierba sana <25 % sugiere considerar resiembra; uniformidad <50 indica posibles zonas con corte irregular; arbolado >15 % genera un mensaje informativo; 'other' >20 % recomienda revisión manual.

8. Etapa 5 — Aplicación web interactiva

Fichero: src/stage5_app.py. La aplicación usa Streamlit, que permite crear interfaces web sin necesidad de utilizar HTML ni JavaScript. Se ejecuta con python3 run_pipeline.py 5 y se abre en <http://localhost:8501>.

8.1 Tab 1: Análisis individual

El usuario sube una imagen y obtiene: métricas principales (Health Score, Nivel de Estrés, Uniformidad, porcentajes de hierba), cuatro vistas (original, segmentación con overlay, mapa de salud, mapa de confianza), gráficas interactivas (gráfico de sectores, velocímetro de salud, barras por zona) y, si se introduce el número de hectáreas, una tabla con el área en m² y ha de cada clase. El botón de descarga genera un PDF con todas las imágenes y métricas.

8.2 Tab 2: Comparación temporal

Permite subir dos imágenes del mismo campo en fechas distintas y comparar la evolución. Se generan las dos segmentaciones lado a lado, un mapa de cambios (rojo = deterioro, verde = mejora) y métricas comparativas con delta (↑↓). Esto permite al greenkeeper hacer un seguimiento mensual y detectar si alguna zona está mejorando o empeorando.

8.3 Sidebar de configuración

Opacidad del overlay (0,1 a 0,9), activación de medición de áreas en hectáreas/m² y leyenda visual de colores por clase.

9. Problemas encontrados y soluciones aplicadas

Nº	Problema	Solución
1	Error SSL al descargar EfficientNet	Ejecutar el script de instalación de certificados SSL incluido con Python.
2	El sistema entraba en reposo durante el entrenamiento	Impedir que el sistema entre en suspensión mientras el proceso de entrenamiento está activo (a través de las opciones de energía del sistema operativo).
3	Hierba sana detectada como estresada	Subir S_min a 55 en stressed_grass y añadir CLAHE para normalizar la iluminación.
4	Franjas de corte detectadas como estrés	smooth_mowing_stripes(): componentes conexas pequeñas rodeadas de hierba sana se reclasifican como sanas.
5	Árboles no detectados o con falsos positivos	Detección adaptativa: semillas (AND estricto) → aprendizaje del color → expansión adaptativa → filtro de área mínima 1.200 px.
6	Árboles penalizaban el health score	La detección de árboles se aplica también a píxeles clasificados como stressed_grass, excluyéndolos del cálculo.

Nº	Problema	Solución
7	Mensaje de malas hierbas incorrecto con muchos árboles	Separar conteo de árboles del conteo de 'other'. El mensaje solo considera 'other' sin árboles.
8	COLORMAP_RdYlGn no existe en OpenCV	Crear manualmente una LUT que va de rojo a amarillo a verde usando interpolación lineal.

10. Resumen de mejoras implementadas

Mejora	Fichero	Motivo
S_min subido en stressed_grass	config.py	Falsos positivos por sombras
CLAHE normalización de luz	stage4_inference.py	Sol confundido con estrés
Refinamiento bunkers por forma	stage4_inference.py	Falsos positivos rectangulares
Corrección franjas de corte	stage4_inference.py	Rayas del cortacésped
Detección adaptativa de árboles	stage4_inference.py	Árboles mal clasificados
Mapa de confianza	stage4_inference.py	Transparencia sobre las predicciones del modelo
LUT rojo-amarillo-verde manual	stage4_inference.py	COLORMAP_RdYlGn inexistente en OpenCV
Tab comparación temporal	stage5_app.py	Seguimiento en el tiempo
Medición áreas en m ²	stage5_app.py	Cuantificar zonas tratables

Mejora	Fichero	Motivo
Descarga de informe PDF	stage5_app.py	Informe profesional descargable
Árboles en leyenda visual	stage5_app.py	Comunicar la nueva clase detectada

11. Resultados obtenidos

Métricas del modelo por clase

Clase	Precisión	Recall	F1-Score
healthy_grass	80 %	96 %	87 %
stressed_grass	85 %	88 %	86 %
bunker	41 %	88 %	56 %
other	100 %	93 %	97 %

Evaluación sobre 103.096 patches de test. El bunker es la clase más difícil —aunque el refinamiento por forma ha mejorado los resultados, la confusión con superficies claras sigue siendo el reto principal del sistema.

Capacidades del sistema completo

Analiza una imagen en 10-30 segundos en CPU, sin GPU.

Detecta 5 tipos de zona automáticamente, sin etiquetado manual.

Se adapta a la iluminación de cada imagen gracias a CLAHE.

Corrige artefactos del patrón de corte del césped automáticamente.

Aprende el color de los árboles de cada imagen de forma adaptativa.

Genera informes PDF descargables con todas las métricas.

Permite comparar la evolución del campo entre dos fechas.

Stack tecnológico utilizado

Tecnología	Uso en el proyecto
Python 3.14	Lenguaje principal del proyecto
PyTorch + TorchVision	Red neuronal y entrenamiento
OpenCV	Procesado de imagen, HSV, morfología
EfficientNet-B0	Arquitectura CNN (transfer learning)
Streamlit	Interfaz web interactiva
albumentations	Aumentación de datos de entrenamiento
Matplotlib / Plotly	Visualizaciones y gráficas interactivas
Pandas	Tablas de datos y métricas